

PX-AI（API）無償版 サービス詳細

パナソニック オペレーショナルエクセレンス(株)
デジタル戦略本部
ビジネスエンゲージメント部
イノベーション共創課

パナソニック デジタル(株)
アナリティクスソリューション統括部
エンタープライズソリューション部

2026年4月3日

変更履歴

No.	版数	変更日	変更箇所（ページ等）	変更内容
1	1.0	2024/7/11	初版作成	—
2	1.1	2024/8/7	[1]API仕様について ① [2]コマンドのサンプルと項目説明 [3]（参考）利用リージョンのモデル別対応表	[1][3]APIのベースURLに使用するAPIバージョン情報はAIモデルバージョン情報とは異なる旨の注釈を追加 [2]サンプルコードをコピー＆ペーストした際に実行エラーとならないよう、以下の内容を修正 ・「”」が全角になっている箇所を半角に修正 ・「¥」が正しい文字コードでコピーできるよう修正
3	2.0	2024/9/30	新API追加に伴う資料更新 [1]API概要 [2]新API（messages形式） [3]現行API（prompt形式）	[1]API概要に新API（messages形式）の公開、移行期間についての説明を追加 [2]新API（messages形式）の仕様、コマンドサンプルを追加 [3]現行API（prompt形式）の仕様、コマンドサンプルの説明を修正
4	2.1	2024/12/16	[1]text-embedding(埋め込みモデル)の対応 [2]エラーコード説明 追加	[1]text-embedding(埋め込みモデル)の仕様、コマンドサンプルを追加 [2]よく発生するエラーに関する説明を追加
5	2.2	2025/2/6	[1]4o-miniモデル対応	[1]新API（messages形式）の仕様に4o-miniモデルを追加
6	2.3	2025/4/1	[1]3.5-turboモデル削除	[1]API仕様から3.5-turboモデルを削除
7	3.0	2025/7/9	[1]4.1-nano、4.1-mini、o3、o4-miniモデル対応 [2]APIバージョンアップ対応(ver.2024-10-21)	[1]新API（messages形式）の仕様に新モデルの仕様を追記 [2]新API（messages形式）の仕様に新バージョンの仕様を追記
8	3.1	2025/10/31	[1]API利用時の注意事項 [2]使用制限のあるパラメータ	[1]利用説明、禁止事項の追記 [2]temperatureの固定値について追記
9	4.0	2026/1/26	全体	PX-AI（API）リニューアルにより全面見直し
10	4.1	2026/2/25	[1]chat completions実行イメージ① [2]text-embedding実行イメージ②	プロキシ情報の記載方法を変更
11	4.2	2026/4/3	表紙、[1]PX-AI（API）無償版 利用規約	社名変更

- [PX-AI（API）無償版について](#)
- [ネットワーク構成について](#)
- [PX-AI（API）無償版 利用規約](#)
- [PX-AI（API）利用時の注意事項](#)
- [提供APIの機能一覧](#)
- [利用可能なLLMモデル・バージョン](#)
- [エラーコード説明](#)
- [API仕様・コマンドサンプル](#)
- [chat completionsコマンドのサンプル・実行イメージ](#)
- [text-embeddingコマンドのサンプル・実行イメージ](#)

PX-AI API
無償版

API概要

■本サービスの位置づけ

- ・本サービスは、LLMをエクセルなどの既存プログラムから呼び出し、その効果や有用性を検証したい方向けたものです。
- ・**個人**での利用を想定し、**“お試し利用”**として提供しています。
- ・業務システムへの組み込みや連携、業務での定常的/組織的な利用などは、別途iFlexなどのサービスをご利用いただくことを想定しています。

■利用料金

- ・本サービスは社内の全ての従業員に対して**無償**で提供されます。
（※2025年12月時点。将来変更の可能性あり）

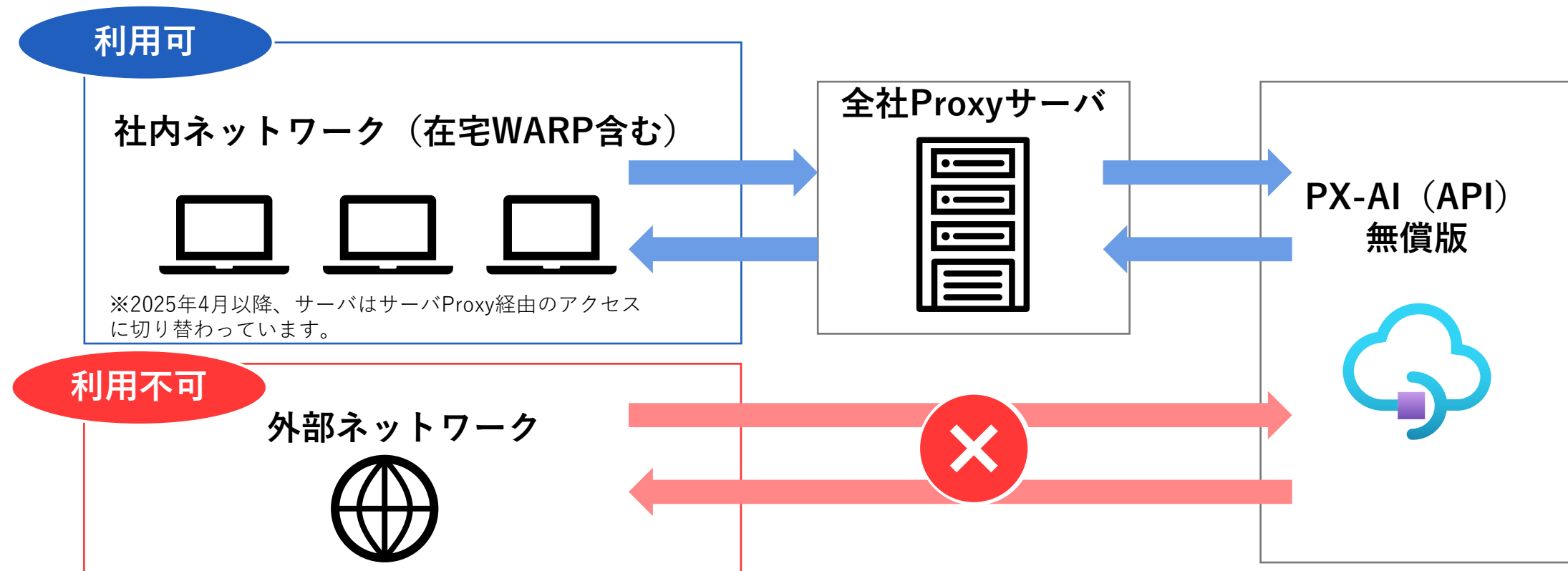
■利用制限

- ・本サービスには、月間の利用回数に制限があります。以下の制限を超えないようご注意ください。
月間利用回数: 100コール/月

※今後、仕様の変更やサービスの停止を予告なく行う場合があります。あらかじめご了承ください。

ネットワーク構成について

PX-AI APIサービス（以下「本サービス」）は社内ネットワークからのみ利用可能です。



1. 提供目的

- i. 本サービスは、パナソニックグループにおける生成AIの業務活用を検証するために公開しているPX-AI（API）の無償版APIサービスです。
- ii. 個人での業務活用/改善検証における利用を想定したサービスで、PX-AI（API）無償版の月間使用回数上限があります。
- iii. サービス利用において、問題が発生した場合は、問題解消の為の情報提供等の協力を依頼する場合があります。

2. 許諾の条件

- i. 本サービスへの入力内容（プロンプト）及び応答内容を自動的に性能検証・向上のために参照・分析することは行っておりません。ただし、お問い合わせ等により検証が必要と判断した場合、必要な範囲でPDにて内容の確認をさせていただくことがあります。

3. 情報セキュリティ

- i. 社内イントラネットワークからのみ利用可能です。

- ii. 情報入力時の留意事項

入力データの二次利用・第三者提供はされないものの、様々なリスクを考慮し、取り扱い可能・不可情報を以下とします。

- **取り扱い可能**

- ✓ **公開情報、当社に帰属するIUO情報、当社に帰属するConfidential情報**→定義は[こちら](#)

- ※24年2月 **日本国内先行**で、IUOまでから、Confidentialまで拡大

- **取り扱い不可**

- ✓ 厳秘(Strictly Confidential)情報、特別管理情報(Confidential-T)、Confidential-P情報(一般人の個人情報、機微な個人情報)

- ✓ IUO情報、Confidential情報のうち、個人情報、他者お預かり情報

- ✓ 他者著作物、誹謗中傷やプライバシー侵害にあたるおそれのある情報等

- iii. 出力情報利用時の留意事項

- **回答は正確とは限りません**、そのまま利用することは避け、人による判断・確認を行ってください。

- 著作権侵害となる判断基準が法的・社会的に明確になっていないため、**潜在的な著作権侵害のリスクがある**ことをご認識の上でご利用ください。

- **出力結果は社内利用に限定**し、そのまま社外に開示・提供することは控えてください。

- 法令により「資格を持った人間が行うこと」が要請されている文書作成などを、資格者の関与なしに行わないでください。

- iv. 生成AIの利用には、法令順守だけでなく、社会受容性の観点から、慎重にお願いします。

3. 情報セキュリティ（続き）

v. データ保存に関連する法的リスク

- 利用しているLLMは一部米国リージョンに設置されています。サーバーが設置されている米国の法律が適用される可能性があるため、米国の法規制に違反しないデータのみを利用するようにしてください。
- 米国の機関によるデータの差し押さえや、これに伴う賠償請求が発生するリスク、サービスが一定期間利用できない状況や、データ内容が米国の機関によって参照される可能性があります。

vi. 利用者の種別や所在国によっては、技術の輸出として輸出管理規制の対象となる場合があるため、技術情報の取り扱いに際しては、必ず各事業場の輸出管理担当者に確認してください。

4. 免責事項

- i. 利用者の誤操作等によるデータの改変や削除についての責任は、PDは一切負うことができません。
- ii. 利用者が入力したプロンプト及び生成AIの回答結果に関してPDは一切責任を負うことができません。
- iii. 利用者は、PDが発行するAPIキーを自己の責任において厳重に管理・保管するものとし、第三者による不正使用を含むいかなる損害についても、PDは一切の責任を負いません。
- iv. APIキーの漏洩、紛失、盗難、または第三者による不正使用に起因して利用者または第三者に生じた損害について、PDは一切の責任を負わないものとします。
- v. 利用者は、APIキーの漏洩または不正使用が判明した場合、直ちにPDに通知し、PDの指示に従うものとします。

ご利用にあたって、以下用途での利用は**禁止**となります。

- ・ 商用利用
- ・ 業務システムとの連携、組み込み
- ・ 稼働保証が常時必要な高いサービスレベルが求められる業務利用（特にSLA等は設けておりません）
- ・ 利用におけるサポートが必須となる利用
（コミュニティポータルサイト経由の問い合わせに対する
ベストエフォートでの対応は可能ですが、随時/即時のサポート等はありません）

PX-AI（API） 無償版で提供するAPIの機能は以下の通りです。

仕様・機能	提供API	提供API
入力形式		chat completions：messages形式 text-embedding：input形式
埋め込みモデル		○ (対応モデル：text-embedding-3-large)
会話履歴の送信		○
画像対応		○ (対応モデル：gpt-4o-mini、gpt-4o、gpt-4.1、gpt-4.1-mini、 gpt-4.1-nano、o3、o4-mini、gpt-5、gpt-5-mini、gpt-5-nano)
OpenAIライブラリ対応		○

利用可能なLLMモデル、バージョン

PX-AI（API）無償版で利用可能なLLMモデルの対応表を紹介します。
サービスを利用する際の参考にしてください。
新たなモデルが提供された際には、内部検討の上随時追加予定です。
※米国リージョンのAzure OpenAI Serviceを使用しています。

デプロイ名	モデル名	GPTモデルのバージョン	システムレート制限
pisc-newsol-openai-uat-gpt-5-nano	gpt-5-nano	2025-08-07	5,000,000
pisc-newsol-openai-uat-gpt-5-mini	gpt-5-mini	2025-08-07	300,000
pisc-newsol-openai-uat-gpt-5	gpt-5	2025-08-07	300,000
pisc-newsol-openai-uat-o4-mini	o4-mini	2025-04-16	300,000
pisc-newsol-openai-uat-o3	o3	2025-04-16	300,000
pisc-newsol-openai-uat-gpt-41-nano	gpt-4.1-nano	2025-04-14	200,000
pisc-newsol-openai-uat-gpt-41-mini	gpt-4.1-mini	2025-04-14	200,000
pisc-newsol-openai-uat-gpt-41	gpt-4.1	2025-04-14	200,000
pisc-newsol-openai-uat-gpt-4o	gpt-4o	2024-08-06	1,000,000
pisc-newsol-openai-uat-gpt-4o-mini	gpt-4o-mini	2024-07-18	2,000,000
pisc-newsol-openai-uat-text-embedding-3-large	text-embedding-3-large	1	210,000

※GPTモデルのバージョンはコール時には使用しません
下記Azure OpenAI Service公式ドキュメントでサポートの記載があるapi-versionを使用ください。

PX-AI（API）無償版利用時によく発生するエラーについてご紹介します。

ステータスコード	PX-AI（API）有償版で出力されるエラー	
	レスポンスに表示されるmessage	内容
0	－（表示なし）	要求の保留中にクライアントにダウンストリーム接続が中止された
400	Your request LLM is not supported	指定したモデル名が利用可能ではない
	Your request operation is not supported	指定したAPIが利用可能ではない
	Invalid JSON format.	リクエストのjsonが壊れている
401	Access denied due to invalid subscription key.	サブスクリプション キー値が無効
		Headerのサブスクリプションキーの書き方がマニュアル通りではない
	Access denied due to missing subscription key.	サブスクリプション キーが指定されていない
403	Forbidden	呼び出し元 IP アドレスが許可されていない（WARP接続されていない）
	Out of call volume quota.	コール数上限を超過している
429	Token limit is exceeded. Try again xx seconds.	1 分間に利用できるトークン量を超過している

※上表に記載のないステータスコードは、Azure OpenAI Serviceで発生するエラーです。
以下の公式サイトをご参照ください。

・ [Microsoft Foundry のドキュメント](#) | [Microsoft Learn](#)

PX-AI API
無償版

API仕様・コマンドサンプル (messages形式・埋め込みモデル)

API仕様について ①

本APIはAzure OpenAI ServiceのAPIを基盤としており、基本的な仕様はAzure OpenAI Serviceに準じますが、一部社内向けにカスタマイズされています。詳細は[Azure公式ドキュメント](#)をご参照ください。

■APIのベースURL

Chat : **{endpoint}/openai/deployments/{deployment-id}/chat/completions?api-version={api-version}**

Embedding : **{endpoint}/openai/deployments/{deployment-id}/embeddings?api-version={api-version}**

{endpoint}	モデル名	{deployment-id}	{api-version}
https://pisc-newsol-openai-uat-mgd.azure-api.net/pxaiapi/2024-06-01 または https://pisc-newsol-openai-uat-mgd.azure-api.net/pxaiapi/2024-10-21	gpt-5-nano	pisc-newsol-openai-uat-gpt-5-nano	{api-version}には、下記公式ドキュメント内の「APIの仕様」でサポートの記載がある「2024-06-01」または「2024-10-21」以前のapi-version※を使用してください。 ※モデル o3、o4-miniを使用するときは、「2024-12-01-preview」以降のモデルを使用してください ※GPTモデルのバージョンとは異なります
	gpt-5-mini	pisc-newsol-openai-uat-gpt-5-mini	
	gpt-5	pisc-newsol-openai-uat-gpt-5	
	o4-mini	pisc-newsol-openai-uat-o4-mini	
	o3	pisc-newsol-openai-uat-o3	
	gpt-4.1-nano	pisc-newsol-openai-uat-gpt-41-nano	
	gpt-4.1-mini	pisc-newsol-openai-uat-gpt-41-mini	
	gpt-4.1	pisc-newsol-openai-uat-gpt-41	
	gpt-4o	pisc-newsol-openai-uat-gpt-4o	
	gpt-4o-mini	pisc-newsol-openai-uat-gpt-4o-mini	
	text-embedding-3-large	pisc-newsol-openai-uat-text-embedding-3-large	

■HTTPメソッド : POST

■リクエストヘッダー

APIを利用するためには事前に発行されたAPIキーが必要です。

- Content-Type: application/json
- api-key:**{APIキー}**

{endpoint}では、api-version「**2024-06-01**」「**2024-10-21**」に対応しています。
新たなapi-versionが提供された際には、内部検討の上随時追加予定です。

Microsoft Foundry Models の Azure OpenAI REST API リファレンス - Azure OpenAI |
Microsoft Learn : <https://learn.microsoft.com/ja-jp/azure/ai-foundry/openai/reference?view=foundry-classic#rest-api-versioning>

■ リクエストボディ

リクエストボディはJSON形式です。パラメータの種類については[Azure公式ドキュメント](#)をご参照ください。
ここでは主なパラメータについて紹介します。

```
{
  "messages": [
    {
      "role": "system",
      "content": "あなたは優秀なAIアシスタントです。"
    },
    {
      "role": "user",
      "content": "猫のお世話の仕方を教えてください。"
    }
  ],
  "max_tokens": 30
}
```

フィールド説明

- **messages**：これまでの会話を構成するメッセージの一覧。role（system、user、assistant）とcontentで構成する。
- **max_tokens**：生成されるテキストの最大トークン数。推論モデル（GPT-5系、o3、o4-mini）は非対応のため、**max_completion_tokens**に置き換えてください。

chat completions :
コマンドプロンプトでテキストを送信する場合

```
curl -x ①http://プロキシ ②https://pisc-newsol-openai-uat-mgd.azure-api.net/pxaiapi/2024-06-01/openai/deployments/③pisc-newsol-openai-uat-gpt-4o/④chat/completions?api-version=⑤2024-06-01 -H "Content-Type: application/json" -H "api-key: ⑥APIキー" -d "{¥"messages¥": [{¥"role¥": ¥"system¥", ¥"content¥": ¥"あなたは優秀なAIアシスタントです。 ¥"}, {¥"role¥": ¥"user¥", ¥"content¥": ¥"こんにちは¥"}] ¥"}"
```

No.	項目の説明
①	経由するプロキシを設定
②	PX-AI APIのEndPointを指定
③	Azure OpenAI Studioでデプロイしたモデルのデプロイ名を指定（※P12「API仕様について ①」を参照）
④	使用する機能を指定：例）chat completions
⑤	Azure OpenAI ServiceのAPIバージョン名を指定（※P12「API仕様について ①」を参照）
⑥	HeaderにPX-AI APIのキー情報を格納（サブスクリプションの主キー）

chat completions実行イメージ①

リクエストとレスポンスの実行イメージを示します。

chat completions :
コマンドプロンプトでテキストを送信する場合

■ リクエスト

```
curl -x http://プロキシ https://pisc-newsol-openai-uat-mgd.azure-api.net/pxaiapi/2024-10-21^  
/openai/deployments/pisc-newsol-openai-uat-gpt-41-nano/chat/completions?api-version=2024-10-21 ^  
-H "Content-Type: application/json" ^  
-H "api-key: APIキー" ^  
-d '{"messages":[{"role": "system", "content": "あなたは優秀なAIアシスタントです。"}, {"role": "user", "content": "こんにちは"}]}'
```

※ -H, -d コマンドの前は半角スペースが入ります。

■ レスポンス

```
{"choices":[{"content_filter_results":{"hate":{"filtered":false,"severity":"safe"},"self_harm":{"filtered":false,"severity":"safe"},"sexual":{"filtered":false,"severity":"safe"},"violence":{"filtered":false,"severity":"safe"},"finish_reason":"stop","index":0,"logprobs":null,"message":{"content":"こんにちは！ どのようにお手伝いしましょうか？","role":"assistant"}}, {"created":1726734115,"id":"chatcmpl-A96iBZqklrUaqPY9zgLpm5lkbvk8P","model":"gpt-41-nano-2025-04-14","object":"chat.completion","prompt_filter_results":{"prompt_index":0,"content_filter_results":{"hate":{"filtered":false,"severity":"safe"},"self_harm":{"filtered":false,"severity":"safe"},"sexual":{"filtered":false,"severity":"safe"},"violence":{"filtered":false,"severity":"safe"}}},"system_fingerprint":"fp_80a1bad4c7","usage":{"completion_tokens":14,"prompt_tokens":24,"total_tokens":38}}]}
```

```
from openai import AzureOpenAI
import os

# --- プロキシ設定 ---
os.environ["HTTP_PROXY"] = "http://プロキシ"
os.environ["HTTPS_PROXY"] = "http://プロキシ"

client = AzureOpenAI(
    azure_endpoint = "ENDPOINT",
    api_key = "API_KEY",
    api_version = "API_VERSION"
)

response = client.chat.completions.create(
    model = "DEPLOYMENT_ID",
    messages = [
        {"role": "system", "content": "あなたは優秀なAIアシスタントです。"},
        {"role": "user", "content": "猫のお世話の仕方を教えてください。"}
    ]
)

# print(response) #json形式 (整形なし)
# print(response.model_dump_json(indent=2)) #json形式
print(response.choices[0].message.content) #回答のみ
```

chat completions : Pythonでテキストを送信する場合

- ・ Azure OpenAIライブラリが「OpenAI Python 1.x」の場合は、左例のようにリクエストを送信します。
- ・ messages内では、system、assistant、userの3つのroleを指定できます。messagesのsystemにAIの役を設定することで、役に沿った回答を返すようになります。
- ・ roleの詳細、その他使用可能なパラメータは下記公式ドキュメントを参照してください。

Work with the GPT-35-Turbo and GPT-4 models - Azure OpenAI Service |
Microsoft Learn : <https://learn.microsoft.com/ja-jp/azure/ai-services/openai/how-to/chatgpt?tabs=python-new>

chat completions : Pythonでテキストを送信する場合

■ リクエスト

```
from openai import AzureOpenAI
import os

# --- プロキシ設定 ---
os.environ["HTTP_PROXY"] = "http://プロキシ"
os.environ["HTTPS_PROXY"] = "http://プロキシ"

client = AzureOpenAI(
    azure_endpoint = "https://pisc-newsol-openai-uat-mgd.azure-
api.net/pxaiapi/2024-10-21",
    api_key = "API_KEY",
    api_version = "2024-10-21"
)

response = client.chat.completions.create(
    model = "pisc-newsol-openai-uat-gpt-41-nano",
    messages = [
        {"role": "system", "content": "あなたは優秀なAIアシスタントです。"},
        {"role": "user", "content": "猫のお世話の仕方を教えてください。"}
    ]
)

print(response.choices[0].message.content) #回答のみ
```

■ レスポンス

猫のお世話にはいくつか基本的なポイントがあります。以下に、猫を飼う上での主なケアの方法をまとめました。

1. 食事管理

- **適切なキャットフード**: 年齢や健康状態に応じて、適切なキャットフードを選んでください。市販のドライフードやウェットフードが一般的ですが、栄養バランスが取れたものを選ぶことが重要です。
- **給水**: 常に新鮮な水を用意しましょう。水飲み場やウォーターボウルを清潔に保つことも大切です。

2. トイレケア

- **トイレの設置**: 猫は通常、自分のトイレを選ぶことができます。家の中に複数のトイレを用意し、猫の数プラス1つが理想です。
- **掃除**: トイレは定期的に掃除し、猫砂を入れ替えましょう。清潔さを保つことで、使いやすくなりトイレのトラブルを防げます。

3. 健康管理

- **定期的な健康診断**: 獣医師による定期的な健康診断を受けることで、病気の早期発見が可能になります。
- **ワクチン接種**: 必要なワクチン接種を行い、寄生虫予防も忘れずに行いましょう。
- **ブラッシング**: 毛の長さに応じて、定期的にブラシをかけて毛玉を防ぎつつ、皮膚状態をチェックします。

4. 環境の整備

- **安全な住環境**: 猫が安全に遊べる環境を整え、危険な物（例えば、食べてはいけない植物や薬品など）を取り除きます。
- **運動と遊び**: キャットタワーやおもちゃを用意し、運動や遊びを促進します。毎日少しの時間を猫と遊ぶことで、ストレス解消やコミュニケーションを図ることができます。

5. 社会性と愛情

- **スキンシップ**: 猫との信頼関係を築くために、優しく触れ合ったり、遊んだりしましょう。猫によって好みは異なりますが、美味しいおやつなどを使って良い体験を提供することも大切です。
- **注意深い観察**: 猫の行動や健康状態に注意を払い、異常がある場合は早めに獣医師に相談しましょう。

これらの基本を守ること、健康で幸せな猫を育てることができます。猫の性格や好みにも配慮しながら、楽しい時間を過ごしてください。

chat completions コマンドのサンプル③

```
from openai import AzureOpenAI
import os

# --- プロキシ設定 ---
os.environ["HTTP_PROXY"] = "http://プロキシ"
os.environ["HTTPS_PROXY"] = "http://プロキシ"

api_base = "ENDPOINT"
api_key = "API_KEY"
deployment_name = "DEPLOYMENT_ID"
api_version = "API_VERSION"

client = AzureOpenAI(
    api_key=api_key,
    api_version=api_version,
    base_url=f"{api_base}/openai/deployments/{deployment_name}"
)

response = client.chat.completions.create(
    model=deployment_name,
    messages=[
        { "role": "system", "content": "あなたは優秀なAIアシスタントです。" },
        { "role": "user", "content": [
            {
                "type": "text",
                "text": "この画像を説明してください"
            },
            {
                "type": "image_url",
                "image_url": {
                    "url": "IMAGE_URL"
                }
            }
        ]
    },
    max_tokens = 1000
)

# print(response) #json形式（整形なし）
# print(response.model_dump_json(indent=2)) #json形式
print(response.choices[0].message.content) #回答のみ
```

chat completions : Python OpenAIライブラリで画像を使用する場合

- ・ "DEPLOYMENT_ID"には、画像を処理できるモデルを使用してください。（例：gpt-4o）
- ・ 画像を送信する場合、パラメータ"max_tokens"を指定しないと応答が途切れる場合があります。
※o3, o4-miniは"max_tokens"が使用不可のため 指定なしで
使用してください。
- ・ 送信する画像は、リクエストごとに10個の制限があります。
- ・ ローカル画像を使用する場合は、base64 に変換することで使用できます。下記公式ドキュメントの変換ツール（Pythonコード）を参考にしてください。

```
"type": "image_url",
"image_url": {
    "url": "data:image/jpeg;base64,IMAGE_URL"
}
```

GPT-4 Turbo with Vision モデルを使用する方法 - Azure OpenAI Service | Microsoft Learn : <https://learn.microsoft.com/ja-jp/azure/ai-services/openai/how-to/gpt-with-vision?tabs=python%2Csystem-assigned%2Cresource#call-the-chat-completion-apis>

■ リクエスト

```
from openai import AzureOpenAI
import os

# --- プロキシ設定 ---
os.environ["HTTP_PROXY"] = "http://プロキシ"
os.environ["HTTPS_PROXY"] = "http://プロキシ"

api_base = "https://pisc-newsol-openai-uat-mgd.azure-api.net/pxaiapi/2024-10-21"
api_key= "API_KEY"
deployment_name = "pisc-newsol-openai-uat-gpt-41-nano"
api_version = "2024-10-21"

client = AzureOpenAI(
    api_key=api_key,
    api_version=api_version,
    base_url=f"{api_base}/openai/deployments/{deployment_name}"
)

response = client.chat.completions.create(
    model=deployment_name,
    messages=[
        { "role": "system", "content": "あなたは優秀なAIアシスタントです。" },
        { "role": "user", "content": [
            {
                "type": "text",
                "text": "この画像を説明してください"
            },
            {
                "type": "image_url",
                "image_url": {
                    "url": "IMAGE_URL"
                }
            }
        ]
    },
    max_tokens = 1000
)

print(response.choices[0].message.content) #回答のみ
```

chat completions :
Python OpenAIライブラリで画像を使用する場合

■ レスポンス

この画像には、「Panasonic」のロゴが写っています。
黒色の「Panasonic」という文字がグレーの背景に配置
されています。Panasonicは、家電製品や電子機器を製
造する日本の大手企業です。

```
import requests
import json
import os

# --- プロキシ設定 ---
os.environ["HTTP_PROXY"] = "http://プロキシ"
os.environ["HTTPS_PROXY"] = "http://プロキシ"

api_base = "ENDPOINT"
API_KEY = "API_KEY"
deployment_name = "DEPLOYMENT_ID"

base_url = f"{api_base}/openai/deployments/{deployment_name}"
headers = {
    "Content-Type": "application/json",
    "api-key": API_KEY
}

endpoint = f"{base_url}/chat/completions?api-version=API_VERSION"
data = {
    "messages": [
        { "role": "system", "content": "あなたは優秀なAIアシスタントです。" },
        { "role": "user", "content": [
            {
                "type": "text",
                "text": "この画像を説明してください"
            },
            {
                "type": "image_url",
                "image_url": {
                    "url": "IMAGE_URL"
                }
            }
        ]
    }
],
    "max_tokens": 2000
}

response = requests.post(endpoint, headers=headers, data=json.dumps(data))

print(f"Status Code: {response.status_code}") #ステータスコード
print(response.text)
```

chat completions : Rest APIで画像を使用する場合

- PythonのRequestモジュールを使用してリクエストを送信します。
- 詳細は、前ページ「コマンドのサンプル② (Python OpenAIライブラリを使って画像を送信する場合)」や下記公式ドキュメントを参照してください。

ビジョン対応チャット モデルを使用する方法 - Azure OpenAI in Microsoft Foundry Models |
Microsoft Learn : <https://learn.microsoft.com/ja-jp/azure/ai-foundry/openai/how-to/gpt-with-vision?view=foundation-classic&tabs=python#call-the-chat-completion-apis>

■ リクエスト

```
import requests
import json
import os

# --- プロキシ設定 ---
os.environ["HTTP_PROXY"] = "http://プロキシ"
os.environ["HTTPS_PROXY"] = "http://プロキシ"

api_base = "https://pisc-newsol-openai-uat-mgd.azure-api.net/pxaiapi/2024-10-21"
API_KEY = "API_KEY"
deployment_name = "pisc-newsol-openai-uat-gpt-41-nano"

base_url = f"{api_base}/openai/deployments/{deployment_name}"
headers = {
    "Content-Type": "application/json",
    "api-key": API_KEY
}

endpoint = f"{base_url}/chat/completions?api-version=2024-10-21"
data = {
    "messages": [
        { "role": "system", "content": "あなたは優秀なAIアシスタントです。" },
        { "role": "user", "content": [
            {
                "type": "text",
                "text": "この画像を説明してください"
            },
            {
                "type": "image_url",
                "image_url": {
                    "url": "IMAGE_URL"
                }
            }
        ]
    },
    "max_tokens": 2000
}

response = requests.post(endpoint, headers=headers, data=json.dumps(data))

print(f"Status Code: {response.status_code}") # ステータスコード
print(response.text)
```

chat completions : Rest APIで画像を使用する場合

■ レスポンス

```
Status Code: 200
{"choices":[{"content_filter_results":{"hate":{"filtered":false,"severity":"safe"},"self_harm":{"filtered":false,"severity":"safe"},"sexual":{"filtered":false,"severity":"safe"},"violence":{"filtered":false,"severity":"safe"},"finish_reason":"stop","index":0,"logprobs":null,"message":{"content":"この画像には「Panasonic」という文字が書かれています。白からグレーへのグラデーションの背景に、黒い文字で「Panasonic」と書かれています。このロゴは、日本の大手電機メーカー、パナソニック株式会社のロゴです","role":"assistant"}},"created":1726729077,"id":"chatcmpl-A95OvjLPqneUJAUZeJpdaUqQWXWXh","model":"gpt-41-nano-2025-04-14","object":"chat.completion","prompt_filter_results":[{"prompt_index":0,"content_filter_result":{"jailbreak":{"filtered":false,"detected":false}},"prompt_index":1,"content_filter_result":{"sexual":{"filtered":false,"severity":"safe"},"violence":{"filtered":false,"severity":"safe"},"hate":{"filtered":false,"severity":"safe"},"self_harm":{"filtered":false,"severity":"safe"}}},"system_fingerprint":"fp_80a1bad4c7","usage":{"completion_tokens":62,"prompt_tokens":114,"total_tokens":176}
],"system_fingerprint":"fp_80a1bad4c7","usage":{"completion_tokens":62,"prompt_tokens":114,"total_tokens":176}]
```

```
from openai import AzureOpenAI
import os

#--- プロキシ設定 ---
os.environ["HTTP_PROXY"] = "http://プロキシ"
os.environ["HTTPS_PROXY"] = "http://プロキシ"

api_base = "ENDPOINT"
api_key= "API_KEY"
deployment_name = "DEPLOYMENT_ID"
api_version = "API_VERSION"

client = AzureOpenAI(
    api_key=api_key,
    api_version=api_version,
    base_url=f"{api_base}/openai/deployments/{deployment_name}"
)

response = client.chat.completions.create(
    model="pisc-newsol-openai-uat-o4-mini",
    messages=[
        {"role": "user", "content": "初めてのPython APIを書くとき、どのようなステップを  
考えるべきでしょうか?"},
    ],
    max_completion_tokens = 1000
)

print(response.choices[0].message.content)
```

chat completions : Python OpenAIライブラリで推論モデルを使用する場合

- “DEPLOYMENT_ID”には、推論思考できるモデルを使用してください。（例：o3）
- 詳細は下記公式ドキュメントをご参照ください。

Azure OpenAI 推論モデル - GPT-5 シリーズ、o3-mini、o1、o1-mini - Azure OpenAI |

Microsoft Learn : <https://learn.microsoft.com/ja-jp/azure/ai-foundry/openai/how-to/reasoning?view=foundation-classic&tabs=csharp%2Cgpt-5>

chat completions : Python OpenAIライブラリで推論モデルを使用する場合

■ リクエスト

```
from openai import AzureOpenAI
import os

#--- プロキシ設定 ---
os.environ["HTTP_PROXY"] = "http://プロキシ"
os.environ["HTTPS_PROXY"] = "http://プロキシ"

api_base = "ENDPOINT"
api_key= "API_KEY"
deployment_name = "DEPLOYMENT_ID"
api_version = "API_VERSION"

client = AzureOpenAI(
    api_key=api_key,
    api_version=api_version,
    base_url=f"{api_base}/openai/deployments/{deployment_name}"
)

response = client.chat.completions.create(
    model="pisc-newsol-openai-uat-o4-mini",
    messages=[
        {"role": "user", "content": "初めてのPython APIを書くとき、どのようなステップを考えるべきでしょうか?"},
    ],
    max_completion_tokens = 3000
)

print(response.choices[0].message.content)
```

■ レスポンス

Python で初めて API を作る際には、以下のような大まかなステップを踏むとスムーズです。

1. 要件定義・設計フェーズ
 - ・ API の目的とユースケースを整理
 - ・ 提供する機能（リソース）とエンドポイント一覧を決める
 - ・ リクエスト／レスポンスのスキーマを定義（JSON フィールド、型、必須／任意など）
 - ・ 認証・認可方式（API キー／JWT／OAuth2 など）やエラーレスポンス仕様の策定→ OpenAPI(Swagger) 仕様でドキュメント化しておくとの共有がラク
2. 開発環境の準備
 - ・ 仮想環境（venv／pyenv+venv／poetry など）を作成
 - ・ requirements.txt または pyproject.toml（Poetry）で依存管理
 - ・ Git リポジトリを作成し、ディレクトリ構成を決める└ 例：
 - └─ app/ （アプリ本体コード）
 - └─ tests/ （テストコード）
 - └─ Dockerfile （コンテナ化するなら）
 - └─ README.md
3. フレームワーク選定・ベース実装
 - ・ 軽量：Flask、FastAPI、Falcon
 - ・ フルスタック：Django REST Framework
 - ・ 特に型チェックや自動ドキュメント機能を重視するなら FastAPI がオススメ└ `uvicorn + FastAPI` で起動、`@app.get` など最低限の Hello World を動かしてみる
4. データモデル・ビジネスロジックの実装
 - ・ DB を使うなら SQLAlchemy、TortoiseORM、Django ORM などを設定
 - ・ Pydantic（FastAPI）や Marshmallow（Flask）で入力検証・シリアライズ
 - ・ サービス層（ビジネスロジック）とルーティング層を分離し、可読性・テスト容易性を確保
5. ルーティングと CRUD 実装
 - ・ RESTful にリソース設計（GET/POST/PUT/PATCH/DELETE）
 - ・ HTTP ステータスコードを適切に返却
 - ・ 例外・エラー処理（404、400、500 など）をハンドリングし、一貫したエラーフォーマットで返却
6. ドキュメント化
 - ・ OpenAPI から自動生成された Swagger UI／Redoc を用意
 - ・ README.md

text-embedding :
Python OpenAIライブラリで埋め込みモデルを使用する場合

```
import os

#--- プロキシ設定 ---
os.environ["HTTP_PROXY"] = "http://プロキシ"
os.environ["HTTPS_PROXY"] = "http://プロキシ"

client = AzureOpenAI(
    api_key = "API_KEY",
    api_version = "API_VERSION",
    azure_endpoint = "END_POINT"
)

response = client.embeddings.create(
    input = "ここにテキストを入力",
    model= "DEPLOYMENT_ID"
)

print(response.model_dump_json(indent=2))
```

- ・ "DEPLOYMENT_ID"には、埋め込みモデルを使用してください。
(現在は"text-embedding-3-large"のみ対応)

- ・ csvファイルを用いて埋め込みモデルを使用する場合は、以下の公式ドキュメントをご参照ください。

チュートリアル: Azure OpenAI サービスの埋め込みとドキュメント検索を調べる方法 - Azure OpenAI Service |

Microsoft Learn : <https://learn.microsoft.com/ja-jp/azure/ai-services/openai/tutorials/embeddings?tabs=python-new%2Ccommand-line&pivots=programming-language-python>

text-embedding実行イメージ①

リクエストとレスポンスの実行イメージを示します。

text-embedding :
Python OpenAIライブラリで埋め込みモデルを使用する場合

■ リクエスト

```
import os

# --- プロキシ設定 ---
os.environ["HTTP_PROXY"] = "http://プロキシ"
os.environ["HTTPS_PROXY"] = "http://プロキシ"

client = AzureOpenAI(
    api_key = "API_KEY",
    api_version = "2024-10-21",
    azure_endpoint = "https://pisc-newsol-openai-uat-mgd.azure-
api.net/pxaiapi/2024-10-21"
)

response = client.embeddings.create(
    input = "a",
    model = "pisc-newsol-openai-uat-text-embedding-3-large"
)

print(response.model_dump_json(indent=2))
```

■ レスポンス

```
{
  "object": "list",
  "data": [
    {
      "object": "embedding",
      "index": 0,
      "embedding": [
        0.021560369,
        ...,
        -0.021166272,
        -0.00851413
      ]
    }
  ],
  "model": "text-embedding-3-large",
  "usage": {
    "prompt_tokens": 9,
    "total_tokens": 9
  }
}
```

text-embedding :
コマンドプロンプトで埋め込みモデルを使用する場合

```
curl -x http://プロキシ ENDPOINT ^  
/openai/deployments/DEPLOYMENT_ID/embeddings?api-version=API_VERSION ^  
-H "Content-Type: application/json" ^  
-H "api-key: API-KEY" ^  
-d "{¥"input¥": ¥"ここにテキストを入力¥"}"
```

- "DEPLOYMENT_ID"には、埋め込みモデルを使用してください。
(現在は"pisc-newsol-openai-uat-text-embedding-3-large"のみ対応)
- csvファイルを用いて埋め込みモデルを使用する場合は、
以下の公式ドキュメントをご参照ください。

Microsoft Foundry Models の埋め込みに関する Azure OpenAI チュートリアル - Azure OpenAI |
Microsoft Learn : <https://learn.microsoft.com/ja-jp/azure/ai-foundry/openai/tutorials/embeddings?view=foundation-classic&tabs=command-line&pivots=programming-language-python>

text-embedding実行イメージ②

リクエストとレスポンスの実行イメージを示します。

text-embedding :
コマンドプロンプトで埋め込みモデルを使用する場合

■ リクエスト

```
curl -x http://プロキシ https://pisc-newsol-openai-uat-mgd.azure-api.net/pxaiapi/2024-10-21^  
/openai/deployments/pisc-newsol-openai-uat-text-embedding-3-large/embeddings?api-  
version=2024-10-21 ^  
-H "Content-Type: application/json" ^  
-H "api-key: API-KEY" ^  
-d "{¥"input¥": ¥"ここにテキストを入力¥"}"
```

■ レスポンス

```
{  
  "object": "list",  
  "data": [  
    {  
      "object": "embedding",  
      "index": 0,  
      "embedding": [  
        0.021560369,  
        ...  
        -0.021166272,  
        -0.00851413  
      ]  
    },  
  ],  
  "model": "text-embedding-3-large",  
  "usage": {  
    "prompt_tokens": 9,  
    "total_tokens": 9  
  }  
}
```